

GLA UNIVERSITY, MATHURA

LESSON DELIVERY PLAN

BCSG0002: COMPUTER PROGRAMMING

B.Tech. Computer Science & Engineering | 3 Credits | L-T-P-J: 3-1-0-0 | 40 Teaching Hours

Prepared & Designed by

Mr. Ajay Dabade

Technical Trainer / Faculty, Department of Computer Engineering & Applications

1. Course Overview

Course Code	BCSG0002	Course Title	Computer Programming
Programme	B.Tech. Computer Science & Engineering	Credits	03
L-T-P-J	3-1-0-0	Total Teaching Hours	40
Module I	20 Hours	Module II	20 Hours
Primary Language	C	Academic Session	2024–25

2. Course Objective

To provide a solid foundation in problem-solving techniques and develop robust programming skills using the C programming language. The course aims to equip students with essential programming constructs and logic that enable them to transition to other programming languages and tackle real-world computational problems.

3. Course Outcomes

CO	Course Outcome
CO1	Demonstrate an understanding of fundamental problem-solving techniques and algorithms.
CO2	Apply the core principles of the C programming language to develop efficient and effective code.
CO3	Manipulate and process data using arrays and strings, implementing basic algorithms for common operations.
CO4	Design and utilize functions to modularize code, enhance readability, and solve complex problems.
CO5	Implement recursive algorithms and understand their applications in problem-solving.
CO6	Utilize pointers to efficiently manage memory and perform advanced data manipulations.
CO7	Understand and apply file handling techniques to perform input and output operations.
CO8	Develop and implement algorithmic solutions to a variety of computational problems, demonstrating the ability to write clean, well-documented, and maintainable code.

4. Lesson Delivery Plan

Lecture No.	Module	Topic	Sub-topic	Learning Objective	Teaching Method	Proposed Activity	Expected Learning Outcome	CO	Bloom's Level	Assessment Method	Reference
1	I	Basics of C	Overview of C, applications and features	Understand the purpose, characteristics and applications of C programming	Interactive Lecture	Discuss real-world applications of C and identify problems that can be solved using programming	Students can explain the role and applications of C programming	CO1, CO2	Understand	Diagnostic questioning / Oral quiz	R1, R3
2	I	Basics of C	Structure of a C Program	Understand the basic structure and components of a C program	Lecture + Live Coding	Analyze a simple C program and identify header, main(), declarations and statements	Students can identify and construct the basic structure of a C program	CO2	Understand, Apply	Code-tracing exercise	R1, R3
3	I	Basics of C	Identifiers, Keywords and Variables	Understand naming rules and use of identifiers, keywords and variables	Interactive Lecture + Demonstration	Identify valid and invalid identifiers and create variables for a given problem	Students can correctly declare and use identifiers and variables	CO2	Understand, Apply	Short coding exercise	R1
4	I	Basics of C	Data Types	Understand basic C data types and their use	Lecture + Demonstration	Write programs using different data types and observe their values	Students can select appropriate data types for a problem	CO2	Understand, Apply	Quiz + coding task	R1, R3
5	I	Basics of C	Formatted Input and Output	Use formatted input and output operations in C	Live Coding + Guided Practice	Create a program that accepts student information and displays it in formatted form	Students can use printf() and scanf() with appropriate format specifiers	CO2	Apply	Programming exercise	R1
6	I	Operators and Expressions	Assignment, Unary and Arithmetic Operators	Apply assignment, unary and arithmetic operators to solve numerical problems	Lecture + Live Coding	Develop programs for arithmetic calculations such as simple interest and average	Students can construct arithmetic expressions correctly	CO2	Apply	Coding exercise	R1, R3
7	I	Operators and Expressions	Relational, Logical and Conditional Operators	Use relational, logical and conditional operators in expressions	Demonstration + Problem Solving	Predict outputs for compound logical expressions and solve decision-based problems	Students can construct and evaluate logical expressions	CO1, CO2	Apply, Analyze	Output prediction exercise	R1
8	I	Operators and Expressions	Bitwise and Special Operators	Understand and apply bitwise and special operators	Interactive Lecture + Demonstration	Perform basic bitwise operations on integer values and analyze results	Students can explain and implement basic bitwise operations	CO2	Understand, Apply	Short quiz + coding task	R1, R2
9	I	Operators and Expressions	Precedence and Associativity	Understand the order of evaluation of operators	Board Work + Code Tracing	Give students complex expressions and ask them to predict results before execution	Students can evaluate expressions using precedence and associativity rules	CO1, CO2	Analyze	Expression-tracing assessment	R1
10	I	Operators and Expressions	IEEE Representation, Lvalue and Rvalue	Understand representation of float and double, and distinguish Lvalue and Rvalue	Conceptual Lecture + Visualization	Analyze sample expressions and classify operands as Lvalue or Rvalue	Students can explain basic representation concepts and distinguish Lvalue/Rvalue	CO2	Understand, Analyze	Conceptual quiz	R1, R2
11	I	Type Conversion	Type Promotion in Expressions	Understand implicit type promotion during expression evaluation	Lecture + Demonstration	Trace expressions involving different data types and predict the resulting type/value	Students can explain type promotion in C expressions	CO2	Understand, Analyze	Code-tracing exercise	R1
12	I	Type Conversion	Assignment Conversion, Truncation and Casting	Apply explicit and implicit type conversion appropriately	Live Coding + Problem Solving	Compare results of implicit conversion and explicit casting using numerical examples	Students can use type casting and identify truncation effects	CO1, CO2	Apply, Analyze	Programming exercise	R1, R3
13	I	Decision and Case Control	if and if-else	Implement basic decision-making using conditional statements	Lecture + Live Coding	Develop programs for positive/negative, even/odd and eligibility-type problems	Students can implement basic conditional logic	CO1, CO2	Apply	Coding assessment	R1, R3
14	I	Decision and Case Control	Nested if-else	Implement multiple levels of decision making	Problem-Based Learning	Solve problems such as finding the largest of three numbers	Students can construct nested conditional solutions	CO1, CO2	Apply, Analyze	Problem-solving exercise	R1

Lecture No.	Module	Topic	Sub-topic	Learning Objective	Teaching Method	Proposed Activity	Expected Learning Outcome	CO	Bloom's Level	Assessment Method	Reference
15	I	Decision and Case Control	switch Statement	Use switch for multi-way decision making	Lecture + Demonstration	Develop a menu-driven calculator using switch	Students can implement menu-driven programs using switch	CO2	Apply	Programming task	R3
16	I	Decision and Case Control	switch versus if-else ladder	Select an appropriate decision structure for a given problem	Comparative Discussion + Problem Solving	Solve the same problem using both switch and if-else and compare approaches	Students can distinguish appropriate use cases for switch and if-else	CO1, CO2	Analyze, Evaluate	Comparative coding exercise	R1, R3
17	I	Loop Control Structure	for Loop	Understand and implement definite repetition using for	Live Coding + Guided Practice	Generate multiplication tables, sums and numerical sequences	Students can design programs using for loops	CO1, CO2	Apply	Coding exercise	R1
18	I	Loop Control Structure	while and do-while	Differentiate and implement entry-controlled and exit-controlled loops	Demonstration + Problem Solving	Implement programs using both while and do-while and compare execution	Students can select and implement suitable loop structures	CO1, CO2	Apply, Analyze	Code-tracing + coding task	R1, R3
19	I	Loop Control Structure	Nested Loops, break, continue and goto	Use nested loops and loop-control statements	Live Coding + Problem-Based Learning	Develop pattern-printing programs and demonstrate break and continue	Students can control loop execution and implement nested loops	CO1, CO2	Apply, Analyze	Programming exercise	R1
20	I	Arrays	1-D Arrays and Operations	Understand array declaration, initialization, address calculation and basic operations	Lecture + Live Coding	Implement insertion, deletion and traversal in a one-dimensional array	Students can declare, initialize and manipulate 1-D arrays	CO3	Apply, Analyze	Coding assessment	R1, R3
21	II	Two-Dimensional Arrays	Declaration, Initialization and Accessing Elements	Understand the representation and manipulation of 2-D arrays	Lecture + Demonstration	Create and display a 2-D array and access individual elements	Students can declare, initialize and access elements of 2-D arrays	CO3	Understand, Apply	Coding exercise	R1
22	II	Two-Dimensional Arrays	Applications and Problem Solving	Apply 2-D arrays to computational problems	Problem-Based Learning	Solve a problem using rows, columns and matrix-style data	Students can apply 2-D arrays to solve computational problems	CO3, CO8	Apply, Analyze	Problem-solving assessment	R1, R3
23	II	Matrix Operations	Matrix Addition and Subtraction	Implement addition and subtraction of matrices	Live Coding + Guided Practice	Implement matrix addition and subtraction for compatible matrices	Students can develop programs for basic matrix operations	CO3	Apply	Programming assignment	R1
24	II	Matrix Operations	Matrix Multiplication	Understand the logic and implementation of matrix multiplication	Step-by-Step Problem Solving	Derive the multiplication algorithm manually and implement it in C	Students can implement matrix multiplication using nested loops	CO3, CO8	Apply, Analyze	Coding assessment	R1, R3
25	II	Matrix Operations	Matrix Transposition	Understand and implement matrix transpose	Demonstration + Coding	Convert a matrix into its transpose and verify the result	Students can implement matrix transposition using 2-D arrays	CO3	Apply	Programming exercise	R1
26	II	Strings	Introduction to Strings	Understand strings as character arrays and their declaration/initialization	Interactive Lecture + Demonstration	Create character arrays and string variables and examine their memory representation	Students can declare and initialize strings in C	CO3	Understand, Apply	Short quiz + coding task	R1, R3
27	II	String Operations	String Length and Copy	Implement string length and copy operations with and without built-in functions	Live Coding + Guided Practice	Implement length and copy operations manually and using library functions	Students can perform string length and copy operations	CO3	Apply	Coding exercise	R1
28	II	String Operations	Reverse and Concatenate	Implement string reversal and concatenation	Problem-Based Learning + Live Coding	Develop programs to reverse a string and concatenate two strings without built-in functions	Students can manipulate strings using character-level operations	CO3, CO8	Apply, Analyze	Programming assignment	R1, R3
29	II	String Operations	String Comparison	Compare strings with and without built-in functions	Demonstration + Code Tracing	Compare manual character-by-character comparison with library-based comparison	Students can compare strings using different approaches	CO3	Apply, Analyze	Coding exercise	R1
30	II	String	1-D and 2-D Character Arrays	Understand declaration, initialization and use of collections of strings	Lecture + Coding Demonstration	Store and display names/words using 1-D and 2-D character arrays	Students can work with character arrays and collections of strings	CO3	Apply	Programming exercise	R1, R3

Lecture No.	Module	Topic	Sub-topic	Learning Objective	Teaching Method	Proposed Activity	Expected Learning Outcome	CO	Bloom's Level	Assessment Method	Reference
31	II	Functions	Function Declaration and Definition	Understand the need for functions and create user-defined functions	Interactive Lecture + Live Coding	Convert a large program into smaller functions	Students can declare, define and call functions	CO4	Understand, Apply	Coding assessment	R1, R3
32	II	Functions	Categories of Functions	Understand different categories of functions based on parameters and return values	Lecture + Demonstration	Implement examples representing different function categories	Students can classify and implement different categories of functions	CO4	Understand, Apply	Short quiz + coding task	R1
33	II	Functions	Parameter Passing — Call by Value	Understand parameter passing using call by value	Code Tracing + Demonstration	Trace changes to formal and actual parameters during function calls	Students can explain and implement call-by-value functions	CO4	Understand, Analyze	Code-tracing assessment	R1, R3
34	II	Functions	Passing Arrays to Functions	Pass arrays to functions and process array data modularly	Live Coding + Problem Solving	Create functions for array operations such as search, sum and sorting	Students can pass arrays to functions and modularize array-based solutions	CO3, CO4	Apply, Analyze	Programming assignment	R1
35	II	Pointers	Pointer Declaration and Initialization	Understand pointer variables, addresses and pointer initialization	Conceptual Lecture + Memory Visualization	Draw memory diagrams showing variables, addresses and pointers	Students can declare and initialize pointer variables	CO6	Understand, Apply	Conceptual quiz	R1, R3
36	II	Pointers	Accessing Variables through Pointers	Understand dereferencing and accessing variables through pointers	Demonstration + Live Coding	Write a program using address-of and dereference operators	Students can access and modify variables through pointers	CO6	Apply	Coding exercise	R1
37	II	Pointers	Arrays, Strings and Pointer Arithmetic	Understand the relationship between arrays, strings and pointers	Visualization + Problem Solving	Traverse an array/string using pointer arithmetic instead of indexing	Students can manipulate arrays and strings using pointers	CO3, CO6	Apply, Analyze	Programming assessment	R1, R3
38	II	Pointers	Pointers to Pointers, Array of Pointers, Pointer to Array, 2-D Arrays and Pointers	Understand advanced pointer relationships and applications	Conceptual Lecture + Code Tracing	Construct memory diagrams and trace advanced pointer expressions	Students can explain and implement advanced pointer structures	CO6	Analyze	Code-tracing + quiz	R1, R2
39	II	Pointers	Function Pointers, Dynamic Memory Allocation, void and NULL Pointers	Understand advanced pointer applications and dynamic memory management	Demonstration + Guided Coding	Demonstrate dynamic allocation and safe use of pointers	Students can explain and apply dynamic memory allocation and advanced pointer concepts	CO6	Apply, Analyze	Coding assessment	R1, R3
40	II	User Defined Types	enum, typedef, Union and Structure	Understand and apply user-defined data types	Lecture + Integrated Problem Solving	Design a simple student/employee record using appropriate user-defined types	Students can define and use enum, typedef, union and structure concepts	CO6, CO8	Understand, Apply, Create	Integrated coding assessment	R1, R3

5. Reference Key

Code	Reference
R1	K. N. King, "C Programming: A Modern Approach", 2nd Edition, W. W. Norton & Company.
R2	K. R. Venugopal & Sudeep R. Prasad, "Mastering C".
R3	Paul Deitel & Harvey Deitel, "C How to Program", 8th Edition, Pearson.
R4	Peter van der Linden, "Expert C Programming: Deep C Secrets", Prentice Hall.
R5	W. Richard Stevens, "Advanced Programming in the UNIX Environment", 3rd Edition, Addison-Wesley Professional.

6. Note on CO Coverage

The supplied curriculum lists CO5 (recursion) and CO7 (file handling) among the Course Outcomes; however, recursion and file handling are not included in the detailed 40-hour Module I/II content. Accordingly, this LDP does not artificially assign separate lectures to those topics. If required by the department, they may be incorporated later as enrichment topics or after verification of the applicable revised curriculum.